

```

# =====
# LOGISTIC REGRESSION - AIRLINE CUSTOMER SATISFACTION
# COMPLETE ANALYSIS - SINGLE CELL
# =====

import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

print("="*60)
print("LOGISTIC REGRESSION - AIRLINE CUSTOMER SATISFACTION")
print("="*60)

# =====
# STEP 1: LOAD AND CLEAN DATA
# =====

print("\nSTEP 1: LOAD AND CLEAN DATA")
print("-"*40)

df = pd.read_csv("617ec7a0-b7f1-423e-b810-23f59803ffb6.csv")

print(f"Dataset: {df.shape[0]} rows, {df.shape[1]} columns")
print(f"Columns: {df.columns.tolist()}")

# Handle missing values
df['Arrival Delay in Minutes'] = df['Arrival Delay in Minutes'].fillna(df['Arrival Delay in Minutes'].mean())

# Feature Engineering - Service Quality Average
service_cols = ['Seat comfort', 'Inflight wifi service', 'Inflight entertainment',
                'Online support', 'Ease of Online booking', 'On-board service',
                'Leg room service', 'Baggage handling', 'Checkin service',
                'Cleanliness', 'Online boarding']
df['Service_Quality_Avg'] = df[service_cols].mean(axis=1)

# Feature Engineering - Total Delay
df['Total Delay'] = df['Departure Delay in Minutes'] + df['Arrival Delay in Minutes']
df['Had_Delay'] = (df['Total Delay'] > 0).astype(int)

print("\nFeature engineering complete. New features added:")
print(f" - Service_Quality_Avg: {df['Service_Quality_Avg'].mean():.2f} average")
print(f" - Total Delay: {df['Total Delay'].mean():.2f} minutes average")
print(f" - Had_Delay: {df['Had_Delay'].mean():.2%} had delays")

```

```

# =====
# STEP 2: ENCODE CATEGORICAL VARIABLES
# =====

print("\nSTEP 2: ENCODE CATEGORICAL VARIABLES")
print("-"*40)

# Encode categorical variables
categorical_cols = ['Customer Type', 'Type of Travel', 'Class']
label_encoders = {}
for col in categorical_cols:
    le = LabelEncoder()
    df[col] = le.fit_transform(df[col].astype(str))
    label_encoders[col] = le
    print(f" {col}: {dict(zip(le.classes_, le.transform(le.classes_)))}")

# Encode target variable
target_encoder = LabelEncoder()
df['satisfaction_encoded'] = target_encoder.fit_transform(df['satisfaction'])
print(f"\nTarget: {dict(zip(target_encoder.classes_, target_encoder.transform(t

# Check target distribution
print(f"\nTarget Distribution:")
print(df['satisfaction'].value_counts())

# =====
# STEP 3: DEFINE FEATURES AND TARGET
# =====

print("\nSTEP 3: DEFINE FEATURES AND TARGET")
print("-"*40)

# Define features
features = ['Customer Type', 'Age', 'Type of Travel', 'Class', 'Flight Distance',
            'Seat comfort', 'Departure/Arrival time convenient', 'Food and drir',
            'Gate location', 'Inflight wifi service', 'Inflight entertainment',
            'Online support', 'Ease of Online booking', 'On-board service',
            'Leg room service', 'Baggage handling', 'Checkin service',
            'Cleanliness', 'Online boarding', 'Total Delay', 'Had_Delay',
            'Service_Quality_Avg']

X = df[features]
y = df['satisfaction_encoded']

print(f"Features: {len(features)}")
print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")

# =====
# STEP 4: SPLIT DATA INTO TRAIN AND TEST SETS
# =====

```

```

print("\nSTEP 4: SPLIT DATA INTO TRAIN AND TEST SETS")
print("-"*40)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print(f"Training set: {X_train.shape[0]} rows")
print(f"Test set: {X_test.shape[0]} rows")

# =====
# STEP 5: SCALE FEATURES
# =====

print("\nSTEP 5: SCALE FEATURES")
print("-"*40)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Feature scaling complete")

# =====
# STEP 6: TRAIN LOGISTIC REGRESSION MODEL
# =====

print("\nSTEP 6: TRAIN LOGISTIC REGRESSION MODEL")
print("-"*40)

log_reg = LogisticRegression(
    C=1.0,
    solver='lbfgs',
    max_iter=1000,
    class_weight='balanced',
    random_state=42
)
log_reg.fit(X_train_scaled, y_train)

print("Model trained successfully")

# =====
# STEP 7: MAKE PREDICTIONS
# =====

print("\nSTEP 7: MAKE PREDICTIONS")
print("-"*40)

y_pred = log_reg.predict(X_test_scaled)
y_pred_proba = log_reg.predict_proba(X_test_scaled)[: , 1]

```

```

print("Predictions complete")

# =====
# STEP 8: EVALUATE MODEL PERFORMANCE
# =====

print("\n" + "="*60)
print("MODEL PERFORMANCE METRICS")
print("="*60)

# Calculate metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
roc_auc = roc_auc_score(y_test, y_pred_proba)

print(f"Accuracy:  {accuracy:.4f} ({accuracy*100:.2f}%)" )
print(f"Precision: {precision:.4f} ({precision*100:.2f}%)" )
print(f"Recall:    {recall:.4f} ({recall*100:.2f}%)" )
print(f"F1-Score:   {f1:.4f} ({f1*100:.2f}%)" )
print(f"ROC-AUC:    {roc_auc:.4f} ({roc_auc*100:.2f}%)" )
print("="*60)

# =====
# STEP 9: CONFUSION MATRIX
# =====

print("\nSTEP 9: CONFUSION MATRIX")
print("-"*40)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
print(f"\nTrue Negatives (Correctly predicted Dissatisfied): {cm[0][0]}")
print(f"False Positives (Dissatisfied predicted as Satisfied): {cm[0][1]}")
print(f"False Negatives (Satisfied predicted as Dissatisfied): {cm[1][0]}")
print(f"True Positives (Correctly predicted Satisfied):      {cm[1][1]}")

# Visualize Confusion Matrix
plt.figure(figsize=(8, 6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Dissatisfied', 'Satisfied'],
            yticklabels=['Dissatisfied', 'Satisfied'])
plt.title('Confusion Matrix - Logistic Regression', fontsize=14)
plt.xlabel('Predicted', fontsize=12)
plt.ylabel('Actual', fontsize=12)
plt.show()

# =====

```

```

# STEP 10: CLASSIFICATION REPORT
# =====

print("\nSTEP 10: CLASSIFICATION REPORT")
print("-"*40)

print("\nClassification Report:")
print(classification_report(y_test, y_pred, target_names=['Dissatisfied', 'Sati

# =====
# STEP 11: ROC CURVE
# =====

print("\nSTEP 11: ROC CURVE")
print("-"*40)

fpr, tpr, thresholds = roc_curve(y_test, y_pred_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'Logistic Regression (AUC = {roc_auc:.3f})', linewidth
plt.plot([0, 1], [0, 1], 'k--', label='Random Classifier', linewidth=1)
plt.xlabel('False Positive Rate', fontsize=12)
plt.ylabel('True Positive Rate', fontsize=12)
plt.title('ROC Curve - Airline Satisfaction Prediction', fontsize=14)
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# =====
# STEP 12: FEATURE IMPORTANCE
# =====

print("\nSTEP 12: FEATURE IMPORTANCE")
print("-"*40)

coefficients = log_reg.coef_[0]
coef_df = pd.DataFrame({
    'Feature': features,
    'Coefficient': coefficients,
    'Odds_Ratio': np.exp(coefficients)
})
coef_df['Abs_Coefficient'] = np.abs(coef_df['Coefficient'])
coef_df = coef_df.sort_values('Abs_Coefficient', ascending=False)

print("\nTOP 10 FEATURES INFLUENCING SATISFACTION:")
print(coef_df[['Feature', 'Coefficient', 'Odds_Ratio']].head(10).to_string(index=

print("\nTOP 5 POSITIVE DRIVERS (Increase Satisfaction):")
for i, (_, row) in enumerate(coef_df[coef_df['Coefficient'] > 0].head(5).iterrows()):
    print(f" {i}. {row['Feature']}: {row['Coefficient']:.4f}")

```

```

print("\nTOP 5 NEGATIVE DRIVERS (Decrease Satisfaction):")
for i, (_, row) in enumerate(coef_df[coef_df['Coefficient'] < 0].head(5).iterrows()):
    print(f" {i}. {row['Feature']}: {row['Coefficient']:.4f}")

# Visualize top coefficients
plt.figure(figsize=(10, 8))
top_features = coef_df.head(15)
colors = ['green' if x > 0 else 'red' for x in top_features['Coefficient']]
plt.barh(top_features['Feature'], top_features['Coefficient'], color=colors)
plt.xlabel('Coefficient Value', fontsize=12)
plt.title('Top 15 Features Influencing Satisfaction', fontsize=14)
plt.axvline(x=0, color='black', linestyle='--', alpha=0.5)
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# =====
# STEP 13: SUMMARY
# =====

print("\n" + "="*60)
print("PROJECT SUMMARY")
print("="*60)

print(f"""
DATASET: {df.shape[0]} rows, {df.shape[1]} columns
TARGET: Satisfaction (Satisfied/Dissatisfied)
MODEL: Logistic Regression

PERFORMANCE:
- Accuracy: {accuracy:.4f} ({accuracy*100:.1f}%)
- Precision: {precision:.4f} ({precision*100:.1f}%)
- Recall: {recall:.4f} ({recall*100:.1f}%)
- F1-Score: {f1:.4f} ({f1*100:.1f}%)
- ROC-AUC: {roc_auc:.4f} ({roc_auc*100:.1f}%)

TOP DRIVERS:
Positive:
1. {coef_df[coef_df['Coefficient'] > 0]['Feature'].iloc[0]}
2. {coef_df[coef_df['Coefficient'] > 0]['Feature'].iloc[1]}
3. {coef_df[coef_df['Coefficient'] > 0]['Feature'].iloc[2]}
Negative:
1. {coef_df[coef_df['Coefficient'] < 0]['Feature'].iloc[0]}
2. {coef_df[coef_df['Coefficient'] < 0]['Feature'].iloc[1]}
3. {coef_df[coef_df['Coefficient'] < 0]['Feature'].iloc[2]}
""")

print("="*60)
print("PROJECT COMPLETED SUCCESSFULLY!")
print("="*60)

```



```

=====
LOGISTIC REGRESSION - AIRLINE CUSTOMER SATISFACTION
=====

STEP 1: LOAD AND CLEAN DATA
-----
Dataset: 129880 rows, 22 columns
Columns: ['satisfaction', 'Customer Type', 'Age', 'Type of Travel', 'Class', 'F1

Feature engineering complete. New features added:
- Service_Quality_Avg: 3.41 average
- Total Delay: 29.76 minutes average
- Had_Delay: 54.19% had delays

STEP 2: ENCODE CATEGORICAL VARIABLES
-----
Customer Type: {'Loyal Customer': np.int64(0), 'disloyal Customer': np.int64(1)
Type of Travel: {'Business travel': np.int64(0), 'Personal Travel': np.int64(1)
Class: {'Business': np.int64(0), 'Eco': np.int64(1), 'Eco Plus': np.int64(2)}

Target: {'dissatisfied': np.int64(0), 'satisfied': np.int64(1)}

Target Distribution:
satisfaction
satisfied      71087
dissatisfied   58793
Name: count, dtype: int64

STEP 3: DEFINE FEATURES AND TARGET
-----
Features: 22
X shape: (129880, 22)
y shape: (129880,)

STEP 4: SPLIT DATA INTO TRAIN AND TEST SETS
-----
Training set: 103904 rows
Test set: 25976 rows

STEP 5: SCALE FEATURES
-----
Feature scaling complete

STEP 6: TRAIN LOGISTIC REGRESSION MODEL
-----
Model trained successfully

STEP 7: MAKE PREDICTIONS
-----
Predictions complete

=====
MODEL PERFORMANCE METRICS
=====
Accuracy: 0.8319 (83.19%)
Precision: 0.8635 (86.35%)

```

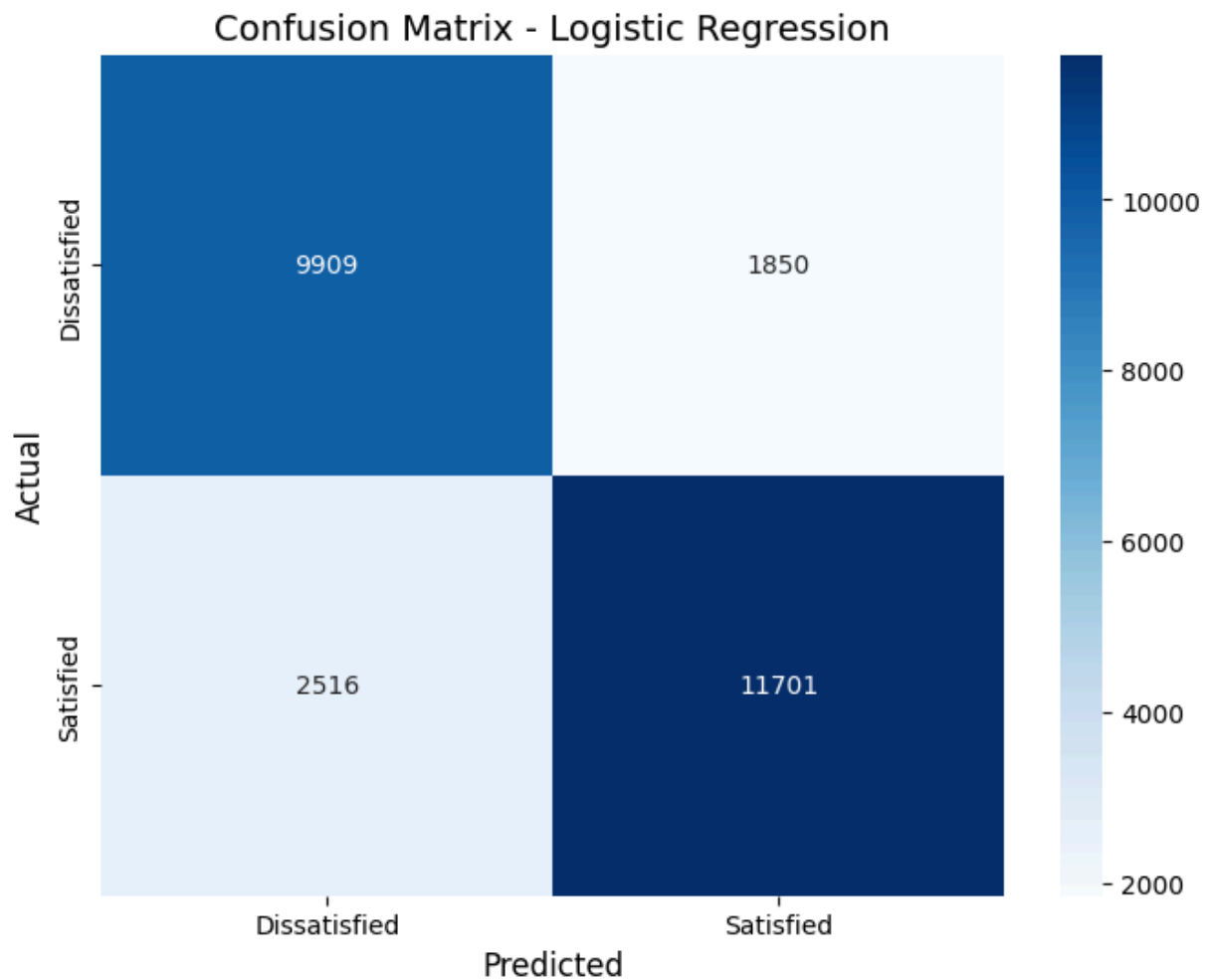

Recall: 0.8230 (82.30%)
F1-Score: 0.8428 (84.28%)
ROC-AUC: 0.9032 (90.32%)

STEP 9: CONFUSION MATRIX

Confusion Matrix:

```
[[ 9909 1850]
 [ 2516 11701]]
```

True Negatives (Correctly predicted Dissatisfied): 9909
False Positives (Dissatisfied predicted as Satisfied): 1850
False Negatives (Satisfied predicted as Dissatisfied): 2516
True Positives (Correctly predicted Satisfied): 11701

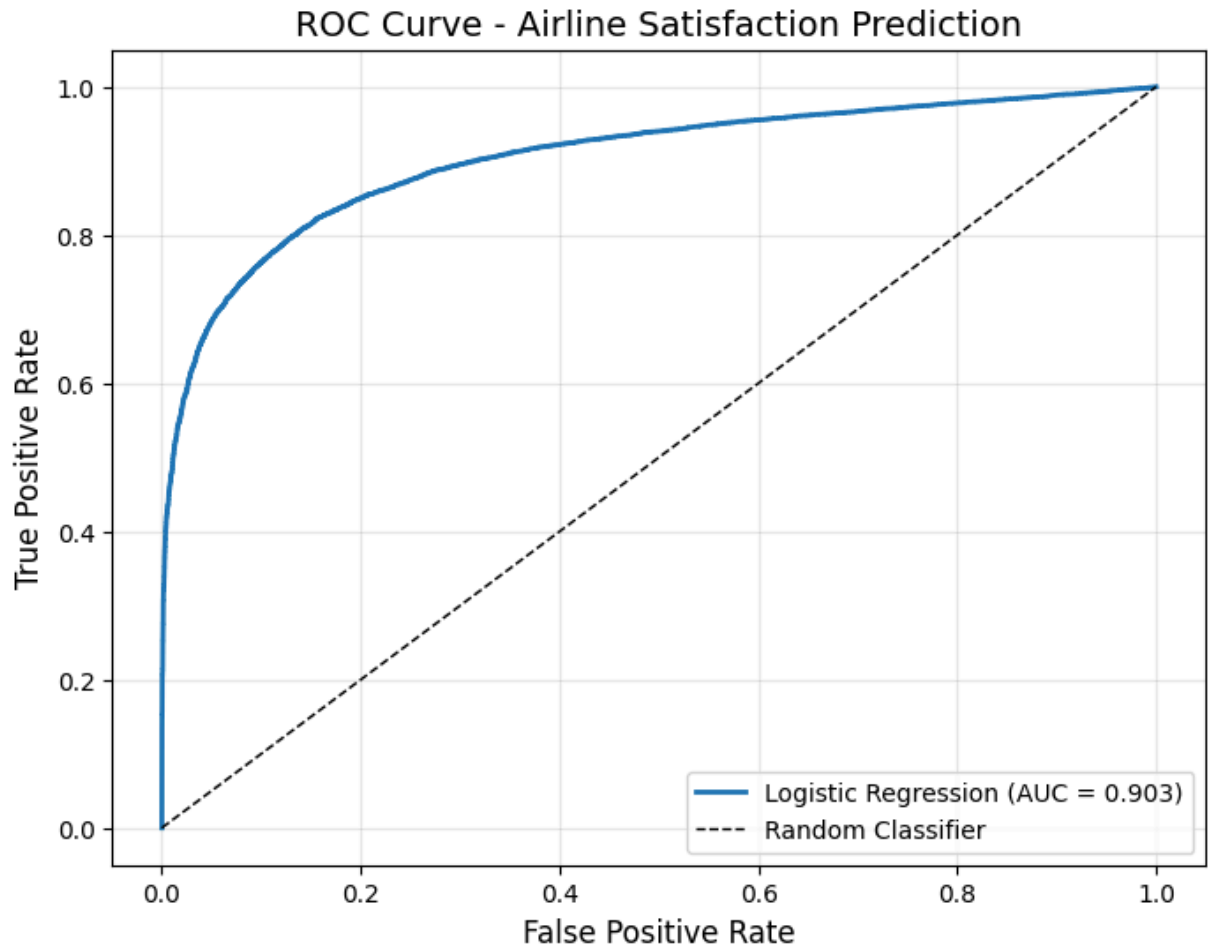


STEP 10: CLASSIFICATION REPORT

Classification Report:

	precision	recall	f1-score	support
Dissatisfied	0.80	0.84	0.82	11759
Satisfied	0.86	0.82	0.84	14217
accuracy			0.83	25976
macro avg	0.83	0.83	0.83	25976
weighted avg	0.83	0.83	0.83	25976

STEP 11: ROC CURVE



STEP 12: FEATURE IMPORTANCE

TOP 10 FEATURES INFLUENCING SATISFACTION:

	Feature	Coefficient	Odds_Ratio
	Inflight entertainment	0.947759	2.579922
	Customer Type	-0.763088	0.466224
	Type of Travel	-0.429050	0.651127
	Service_Quality_Avg	0.402527	1.495600
	On-board service	0.337050	1.400809
Departure/Arrival	time convenient	-0.332072	0.717436
	Checkin service	0.300478	1.350504
	Seat comfort	0.294630	1.342629
	Class	-0.293338	0.745770
	Food and drink	-0.277754	0.757483

TOP 5 POSITIVE DRIVERS (Increase Satisfaction):

1. Inflight entertainment: 0.9478
2. Service_Quality_Avg: 0.4025
3. On-board service: 0.3370
4. Checkin service: 0.3005
5. Seat comfort: 0.2946